

Practical – Week7

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;
static int static_var = 20;

void code_function() {
    printf("Code address    : %p\n", (void *)code_function);
}

int main() {
    int local_var = 30;
    static int static_local = 40;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 50;

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Static local   : %p\n", (void *)&static_local);
    printf("Heap address   : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&local_var);

    code_function();

    free(heap_var);

    return 0;
}
```

```
kavya_aluru@Pandu:~$ nano memory.c
kavya_aluru@Pandu:~$ gcc memory.c -o memory
kavya_aluru@Pandu:~$ ./memory
Global address : 0x5c720b170010
Static address : 0x5c720b170014
Static local   : 0x5c720b170018
Heap address    : 0x5c724a939010
Stack address   : 0x7ffed0da8e3c
Code address    : 0x5c720b16d1a9
```

Report: The C program successfully prints the addresses of code, global, static, heap, and stack variables. The experiment demonstrates the Linux process address space and how different memory segments store and manage data during program execution.

2. Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

int main() {
    int local_var = 20;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 30;

    printf("Process ID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address: %p\n", (void *)&local_var);
    printf("Heap address: %p\n", (void *)heap_var);

    printf("Process is running...\n");

    while (1) {
        sleep(10);
    }

    free(heap_var);
    return 0;
}
```

```
kavya_aluru@Pandu:~$ cd OS_Lab
kavya_aluru@Pandu:~/OS_Lab$ nano memory.c
kavya_aluru@Pandu:~/OS_Lab$ gcc memory.c -o memory
kavya_aluru@Pandu:~/OS_Lab$ ./memory &
[1] 676
Process ID: 676
Global address: 0x64c7dcc55010
Stack address: 0x7ffe8812d59c
Heap address: 0x64c80b0a7010
Process is running...
```

```
kavya_aluru@Pandu:~/OS_Lab$ grep memory /proc/$PID/maps
64c7dcc51000-64c7dcc52000 r--p 00000000 08:30 40074 /ho
me/kavya_aluru/OS_Lab/memory
64c7dcc52000-64c7dcc53000 r-xp 00001000 08:30 40074 /ho
me/kavya_aluru/OS_Lab/memory
64c7dcc53000-64c7dcc54000 r--p 00002000 08:30 40074 /ho
me/kavya_aluru/OS_Lab/memory
64c7dcc54000-64c7dcc55000 r--p 00002000 08:30 40074 /ho
me/kavya_aluru/OS_Lab/memory
64c7dcc55000-64c7dcc56000 rw-p 00003000 08:30 40074 /ho
me/kavya_aluru/OS_Lab/memory
```

```
kavya_aluru@Pandu:~/OS_Lab$ grep '\[heap\]' /proc/$PID/maps
64c80b0a7000-64c80b0c8000 rw-p 00000000 00:00 0 [heap]
kavya_aluru@Pandu:~/OS_Lab$ grep '\[stack\]' /proc/$PID/maps
7ffe8810e000-7ffe8812f000 rw-p 00000000 00:00 0 [stack]
kavya_aluru@Pandu:~/OS_Lab$ grep '\.so' /proc/$PID/maps
78e036800000-78e036828000 r--p 00000000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
78e036828000-78e0369c0000 r-xp 00028000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
78e0369c0000-78e036a0e000 r--p 001c0000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
78e036a0e000-78e036a12000 r--p 0020d000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
78e036a12000-78e036a14000 rw-p 00211000 08:30 13647 /usr/lib/x86_64-linux-gnu/libc.so.6
78e036a14000-78e036aa8000 r--p 00000000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
78e036aa8000-78e036ad7000 r-xp 00001000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
78e036ad7000-78e036ae2000 r--p 00030000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
78e036ae2000-78e036ae4000 r--p 0003b000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
78e036ae4000-78e036ae5000 rw-p 0003d000 08:30 13644 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
```

Report: The `/proc/[PID]/maps` file helps us understand how Linux organizes a process's virtual memory. Memory analysis tools such as `pmmap`, `ps`, and `top` help inspect memory usage and mappings of a running process.